

ADS508_Final

April 13, 2026

1 J&F Realty Insighters Final Project Notebook

1.0.1 ADS508 Data Science with Cloud Computing

Authors: Jun Sik Ryu and Faye Shawntel Corprew

1.1 Project Overview

J&F Realty Insighters is a real estate analytics company focused on identifying the factors that influence housing price trends across metropolitan areas.

The goal of this project is to predict the Zillow Home Value Index, or ZHVI, using housing supply, demand, and prior trend indicators sourced from Zillow datasets and processed in an AWS based workflow.

1.1.1 Business Problem

Traditional real estate analysis often relies on manual trend review and expert intuition. Because housing prices are influenced by multiple interacting factors, a machine learning pipeline can provide more scalable and consistent decision support for identifying promising markets.

1.1.2 Notebook Roadmap

This notebook is organized as a single end to end workflow:

1. Load and inspect the prepared dataset
2. Review data quality and exploratory findings
3. Select features for modeling
4. Train a baseline model and a Random Forest model
5. Evaluate predictive performance
6. Interpret feature importance
7. Summarize findings, limitations, and future enhancements

1.2 AWS and Data Pipeline Context

The broader project pipeline used the following cloud workflow:

- Raw Zillow data stored in Amazon S3
- Exploration and notebook development in Amazon SageMaker Studio
- Transformation from wide format to long format during data preparation
- Feature engineering using lagged and percentage change variables
- Model training using Python and scikit learn in SageMaker

For the final consolidated notebook below, the prepared modeling dataset is loaded directly from a local file when available, with an AWS S3 path included as a fallback reference.

```
[26]: import warnings
warnings.filterwarnings("ignore")
import math
from pathlib import Path
import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.ensemble import RandomForestRegressor
from sklearn.dummy import DummyRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

1.3 Load Final Prepared Dataset

The project's final prepared dataset combines the following Zillow based data sources:

- Zillow Home Value Index
- Housing Inventory
- Market Temperature Index
- Median Sale Price

The transformed dataset already includes time based variables and engineered lag features created in the earlier module notebooks.

```
[27]: LOCAL_DATA_PATH = r"C:
↳\Users\imss\Desktop\USD\2026\ADS508\Final\ADS508_Final\Prepared\full_prepared_dataset.
↳CSV"
```

```

S3_DATA_PATH_1 = "s3://sagemaker-studio-mru71pwhjb/prepared/
↳full_prepared_dataset.csv"
S3_DATA_PATH_2 = "s3://ads508-housing-data-faye/prepared/full_prepared_dataset.
↳csv"

if Path(LOCAL_DATA_PATH).exists():
    data_path = LOCAL_DATA_PATH
elif S3_DATA_PATH_1:
    data_path = S3_DATA_PATH_1
else:
    data_path = S3_DATA_PATH_2

df = pd.read_csv(data_path)
df["Date"] = pd.to_datetime(df["Date"], errors="coerce")

print(f"Loaded data from: {data_path}")
print(f"Shape: {df.shape}")

```

Loaded data from: C:\Users\imss\Desktop\USD\2026\ADS508\Final\ADS508_Final\Prepared\full_prepared_dataset.csv
Shape: (82786, 17)

1.4 Data Dictionary Snapshot

The key variables used in the final modeling dataset are:

- **ZHVI:** Zillow Home Value Index and final target variable
- **Inventory:** Number of homes listed for sale
- **MarketTemp:** Zillow market competitiveness indicator
- **MedianSalePrice:** Observed sale price measure used as a contextual market feature
- **Year, Month:** Time based features
- **ZHVI_Lag1, Inventory_Lag1, MarketTemp_Lag1:** Prior period values used to capture temporal dependence
- **ZHVI_PctChange, Inventory_PctChange:** Momentum style features reflecting recent relative change

[28]: df.info()

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 82786 entries, 0 to 82785
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -

```

```

0   RegionID          82786 non-null int64
1   SizeRank          82786 non-null int64
2   RegionName        82786 non-null object
3   RegionType        82786 non-null object
4   StateName         82786 non-null object
5   Date              82786 non-null datetime64[ns]
6   ZHVI              82786 non-null float64
7   Inventory         82750 non-null float64
8   MarketTemp        82202 non-null float64
9   MedianSalePrice   14308 non-null float64
10  Year              82786 non-null int64
11  Month             82786 non-null int64
12  ZHVI_Lag1         82786 non-null float64
13  Inventory_Lag1    82786 non-null float64
14  MarketTemp_Lag1   82786 non-null float64
15  ZHVI_PctChange    82786 non-null float64
16  Inventory_PctChange 82750 non-null float64
dtypes: datetime64[ns](1), float64(9), int64(4), object(3)
memory usage: 10.7+ MB

```

```
[29]: df.describe(include="all").T.head(20)
```

```

[29]:
count unique      top  freq \
RegionID      82786.0   NaN      NaN   NaN
SizeRank      82786.0   NaN      NaN   NaN
RegionName      82786    894  Zanesville, OH    94
RegionType      82786     1      msa  82786
StateName      82786    50       TX   6152
Date           82786    NaN      NaN   NaN
ZHVI           82786.0   NaN      NaN   NaN
Inventory      82750.0   NaN      NaN   NaN
MarketTemp     82202.0   NaN      NaN   NaN
MedianSalePrice 14308.0   NaN      NaN   NaN
Year           82786.0   NaN      NaN   NaN
Month          82786.0   NaN      NaN   NaN
ZHVI_Lag1      82786.0   NaN      NaN   NaN
Inventory_Lag1  82786.0   NaN      NaN   NaN
MarketTemp_Lag1 82786.0   NaN      NaN   NaN
ZHVI_PctChange  82786.0   NaN      NaN   NaN
Inventory_PctChange 82750.0   NaN      NaN   NaN

```

```

mean min \
RegionID      412458.03282  394297.0
SizeRank      457.972652    1.0
RegionName      NaN      NaN
RegionType      NaN      NaN
StateName      NaN      NaN

```

Date	2022-03-23 09:46:03.858623232	2018-04-30 00:00:00
ZHVI	241769.74908	46486.936299
Inventory	1238.785982	10.0
MarketTemp	54.168877	-84.0
MedianSalePrice	317865.741424	84562.25
Year	2021.678557	2018.0
Month	6.591223	1.0
ZHVI_Lag1	240604.192012	46486.936299
Inventory_Lag1	1241.149252	10.0
MarketTemp_Lag1	54.063501	-87.0
ZHVI_PctChange	0.00499	-0.053176
Inventory_PctChange	-0.000328	-0.358056

	25%	50%	\
RegionID	394548.0	394798.0	
SizeRank	228.0	455.0	
RegionName	NaN	NaN	
RegionType	NaN	NaN	
StateName	NaN	NaN	
Date	2020-03-31 00:00:00	2022-03-31 00:00:00	
ZHVI	148692.101704	198673.747285	
Inventory	154.0	317.0	
MarketTemp	41.0	52.0	
MedianSalePrice	203722.9375	268410.0	
Year	2020.0	2022.0	
Month	4.0	7.0	
ZHVI_Lag1	147926.655854	197698.85576	
Inventory_Lag1	154.0	318.0	
MarketTemp_Lag1	41.0	52.0	
ZHVI_PctChange	0.000931	0.004298	
Inventory_PctChange	-0.045299	0.0	

	75%	max	std
RegionID	395046.0	753929.0	77710.414749
SizeRank	684.0	939.0	267.041342
RegionName	NaN	NaN	NaN
RegionType	NaN	NaN	NaN
StateName	NaN	NaN	NaN
Date	2024-02-29 00:00:00	2026-01-31 00:00:00	NaN
ZHVI	283446.903424	1596557.24204	153473.163156
Inventory	786.0	97260.0	3923.54888
MarketTemp	66.0	240.0	22.451946
MedianSalePrice	377461.95	1661250.0	174918.675412
Year	2024.0	2026.0	2.26785
Month	10.0	12.0	3.43465
ZHVI_Lag1	282100.776392	1596557.24204	152873.725695
Inventory_Lag1	788.0	97260.0	3931.201635

MarketTemp_Lag1	66.0	240.0	22.548387
ZHVI_PctChange	0.008558	0.168843	0.007075
Inventory_PctChange	0.04177	1.374233	0.071576

1.5 Data Quality Review

Confirm that the final prepared file is structurally sound before modeling.

```
[30]: missing_summary = (
        df.isnull()
        .sum()
        .sort_values(ascending=False)
        .rename("MissingCount")
        .to_frame()
    )
    missing_summary["MissingPercent"] = (missing_summary["MissingCount"] / len(df)) * 100
    missing_summary.head(20)
```

```
[30]:
```

	MissingCount	MissingPercent
MedianSalePrice	68478	82.716884
MarketTemp	584	0.705433
Inventory_PctChange	36	0.043486
Inventory	36	0.043486
RegionID	0	0.000000
StateName	0	0.000000
RegionType	0	0.000000
SizeRank	0	0.000000
RegionName	0	0.000000
Date	0	0.000000
ZHVI	0	0.000000
Month	0	0.000000
Year	0	0.000000
ZHVI_Lag1	0	0.000000
Inventory_Lag1	0	0.000000
MarketTemp_Lag1	0	0.000000
ZHVI_PctChange	0	0.000000

The missingness pattern shows that `MedianSalePrice` has substantial gaps, while most engineered features are complete after the preparation workflow.

This matches the earlier project observation that some Zillow series had incomplete coverage across metros and dates.

```
[31]: df = df.dropna(subset=["ZHVI"]).copy()
    df = df.sort_values(["RegionName", "Date"]).copy()
    df = df.dropna(subset=["Inventory", "MarketTemp"]).copy()
    df = df.dropna(subset=["MedianSalePrice"]).copy()
```

```
print(df.isnull().sum())
print(df.shape)
```

```
RegionID          0
SizeRank          0
RegionName        0
RegionType        0
StateName         0
Date              0
ZHVI              0
Inventory          0
MarketTemp        0
MedianSalePrice   0
Year              0
Month             0
ZHVI_Lag1         0
Inventory_Lag1     0
MarketTemp_Lag1   0
ZHVI_PctChange    0
Inventory_PctChange 0
dtype: int64
(14305, 17)
```

```
[32]: print("Unique regions:", df["RegionName"].nunique())
      print("Unique states:", df["StateName"].nunique())
      print("Date range:", df["Date"].min(), "to", df["Date"].max())
```

```
Unique regions: 161
Unique states: 42
Date range: 2018-04-30 00:00:00 to 2026-01-31 00:00:00
```

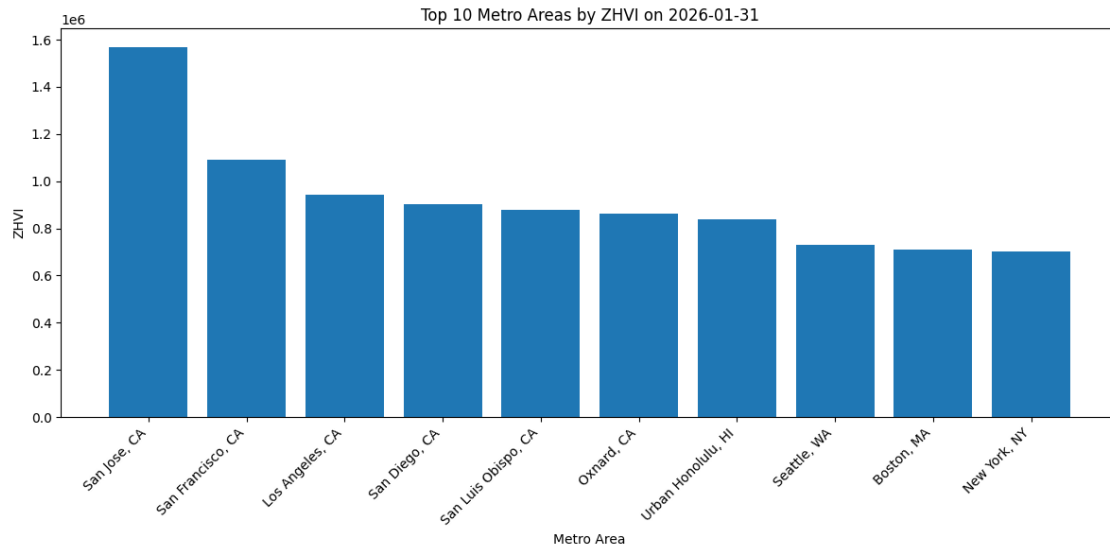
1.6 Exploratory Data Analysis

The final notebook follows the example project style by including a small number of focused visuals that help explain the business problem rather than overwhelming the reader with many disconnected plots.

```
[33]: latest_date = df["Date"].max()
      top_10_latest = (
          df.loc[df["Date"] == latest_date, ["RegionName", "ZHVI"]]
              .dropna()
              .sort_values("ZHVI", ascending=False)
              .head(10)
      )

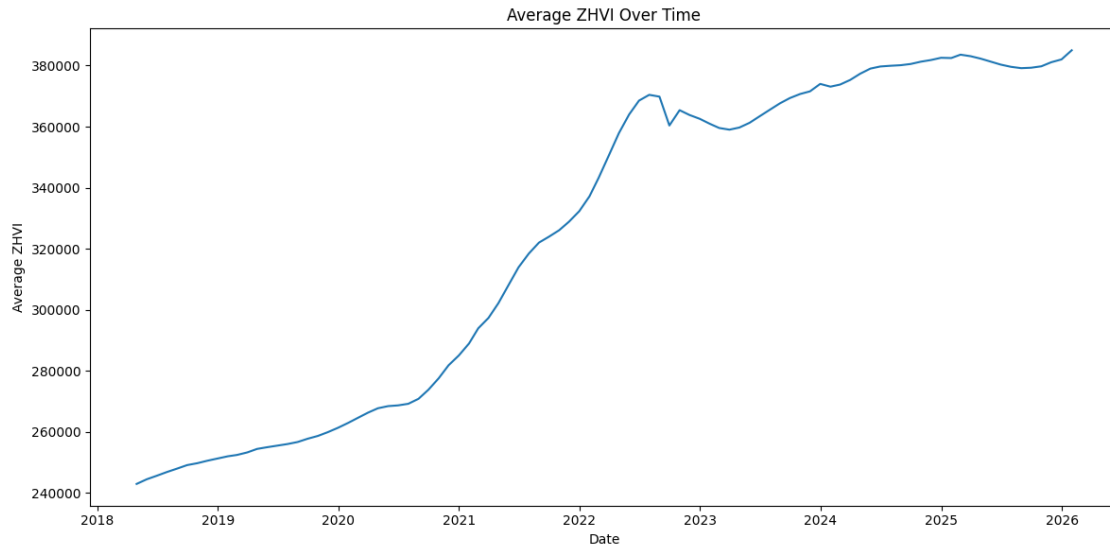
      plt.figure(figsize=(12, 6))
      plt.bar(top_10_latest["RegionName"], top_10_latest["ZHVI"])
      plt.xticks(rotation=45, ha="right")
```

```
plt.title(f"Top 10 Metro Areas by ZHVI on {latest_date.date()}")
plt.xlabel("Metro Area")
plt.ylabel("ZHVI")
plt.tight_layout()
plt.show()
```



```
[34]: monthly_trend = (
    df.groupby("Date")[["ZHVI", "Inventory", "MarketTemp"]]
      .mean()
      .reset_index()
      .sort_values("Date")
)

plt.figure(figsize=(12, 6))
plt.plot(monthly_trend["Date"], monthly_trend["ZHVI"])
plt.title("Average ZHVI Over Time")
plt.xlabel("Date")
plt.ylabel("Average ZHVI")
plt.tight_layout()
plt.show()
```

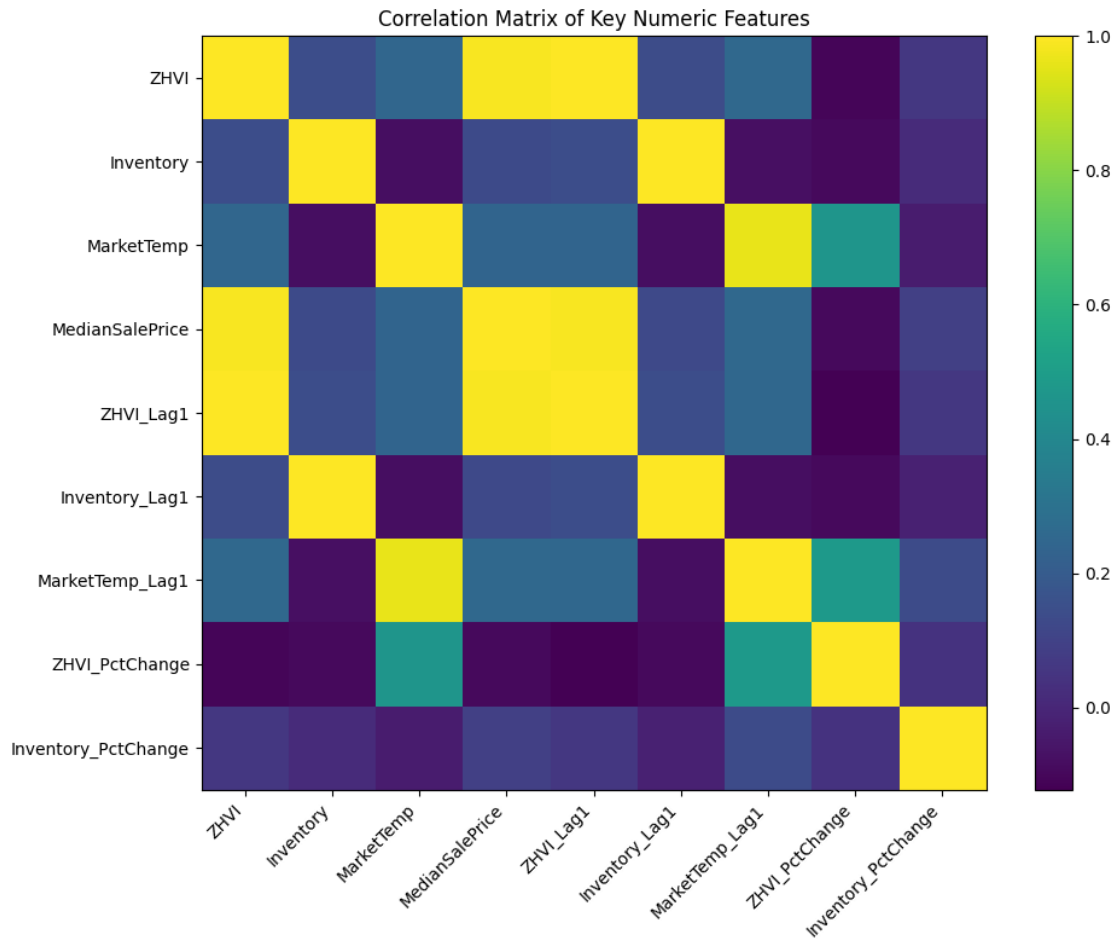



```
[35]: corr_cols = [
        "ZHVI", "Inventory", "MarketTemp", "MedianSalePrice",
        "ZHVI_Lag1", "Inventory_Lag1", "MarketTemp_Lag1",
        "ZHVI_PctChange", "Inventory_PctChange"
    ]

    corr_matrix = df[corr_cols].corr(numeric_only=True)

    plt.figure(figsize=(10, 8))
    plt.imshow(corr_matrix, aspect="auto")
    plt.xticks(range(len(corr_cols)), corr_cols, rotation=45, ha="right")
    plt.yticks(range(len(corr_cols)), corr_cols)
    plt.title("Correlation Matrix of Key Numeric Features")
    plt.colorbar()
    plt.tight_layout()
    plt.show()

    corr_matrix
```



[35]:

	ZHVI	Inventory	MarketTemp	MedianSalePrice	\
ZHVI	1.000000	0.141454	0.248243	0.990666	
Inventory	0.141454	1.000000	-0.081449	0.128920	
MarketTemp	0.248243	-0.081449	1.000000	0.237457	
MedianSalePrice	0.990666	0.128920	0.237457	1.000000	
ZHVI_Lag1	0.999857	0.142335	0.241415	0.990247	
Inventory_Lag1	0.139975	0.997976	-0.079046	0.126075	
MarketTemp_Lag1	0.259045	-0.077996	0.963325	0.256931	
ZHVI_PctChange	-0.107762	-0.092144	0.462148	-0.095660	
Inventory_PctChange	0.059441	0.016451	-0.038760	0.091642	

	ZHVI_Lag1	Inventory_Lag1	MarketTemp_Lag1	\
ZHVI	0.999857	0.139975	0.259045	
Inventory	0.142335	0.997976	-0.077996	
MarketTemp	0.241415	-0.079046	0.963325	
MedianSalePrice	0.990247	0.126075	0.256931	
ZHVI_Lag1	1.000000	0.140869	0.251713	

Inventory_Lag1	0.140869	1.000000	-0.081657
MarketTemp_Lag1	0.251713	-0.081657	1.000000
ZHVI_PctChange	-0.122371	-0.091838	0.486872
Inventory_PctChange	0.058306	-0.020010	0.133367

	ZHVI_PctChange	Inventory_PctChange
ZHVI	-0.107762	0.059441
Inventory	-0.092144	0.016451
MarketTemp	0.462148	-0.038760
MedianSalePrice	-0.095660	0.091642
ZHVI_Lag1	-0.122371	0.058306
Inventory_Lag1	-0.091838	-0.020010
MarketTemp_Lag1	0.486872	0.133367
ZHVI_PctChange	1.000000	0.043650
Inventory_PctChange	0.043650	1.000000

1.6.1 Exploratory Findings

The exploratory review supports three main observations:

1. Housing values vary substantially across metro areas, indicating strong regional heterogeneity.
2. Time based behavior is visible in the aggregated ZHVI trend, which supports inclusion of lag and temporal features.
3. Sale price and lagged home value features appear strongly associated with current ZHVI, suggesting that recent market history carries substantial predictive signal.

1.7 Feature Selection and Modeling Setup

The final notebook is aligned to the design document by using **ZHVI** as the prediction target.

1.7.1 Modeling Choice

A Random Forest Regressor was selected because it can:

- capture nonlinear relationships in tabular housing data
- handle mixed feature types through a preprocessing pipeline
- provide feature importance estimates
- perform well without explicit feature scaling

A simple baseline model is also included so the final model can be compared against a naive benchmark.

```
[36]: feature_cols = [
        "StateName",
        "Year",
```

```

    "Month",
    "Inventory",
    "MarketTemp",
    "MedianSalePrice",
    "ZHVI_Lag1",
    "Inventory_Lag1",
    "MarketTemp_Lag1",
    "ZHVI_PctChange",
    "Inventory_PctChange"
]

target_col = "ZHVI"

df_model = df.dropna(subset=[target_col]).copy()

X = df_model[feature_cols]
y = df_model[target_col]

print("Feature matrix shape:", X.shape)
print("Target vector shape:", y.shape)

```

Feature matrix shape: (14305, 11)
Target vector shape: (14305,)

```

[37]: X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )

print("Training shape:", X_train.shape, y_train.shape)
print("Testing shape:", X_test.shape, y_test.shape)

```

Training shape: (11444, 11) (11444,)
Testing shape: (2861, 11) (2861,)

```

[38]: numeric_features = [c for c in feature_cols if c != "StateName"]
    categorical_features = ["StateName"]

preprocessor = ColumnTransformer(
    transformers=[
        (
            "num",
            Pipeline([
                ("imputer", SimpleImputer(strategy="median"))
            ]),
            numeric_features
        ),
        (
            "cat",

```

```

        Pipeline([
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore"))
        ]),
        categorical_features
    )
]
)

baseline_model = Pipeline([
    ("preprocessor", preprocessor),
    ("regressor", DummyRegressor(strategy="mean"))
])

rf_model = Pipeline([
    ("preprocessor", preprocessor),
    ("regressor", RandomForestRegressor(
        n_estimators=200,
        max_depth=12,
        random_state=42,
        n_jobs=-1
    ))
])

```

1.8 Train Baseline and Random Forest Models

```

[39]: baseline_model.fit(X_train, y_train)
      baseline_preds = baseline_model.predict(X_test)

      rf_model.fit(X_train, y_train)
      rf_preds = rf_model.predict(X_test)

      print("Training complete.")

```

Training complete.

```

[40]: def regression_metrics(y_true, y_pred, model_name):
      return pd.DataFrame({
          "Model": [model_name],
          "MAE": [mean_absolute_error(y_true, y_pred)],
          "RMSE": [math.sqrt(mean_squared_error(y_true, y_pred))],
          "R2": [r2_score(y_true, y_pred)]
      })

      results_df = pd.concat([
          regression_metrics(y_test, baseline_preds, "Baseline Mean Model"),
          regression_metrics(y_test, rf_preds, "Random Forest")
      ], ignore_index=True)

```

```
results_df
```

```
[40]:
```

	Model	MAE	RMSE	R2
0	Baseline Mean Model	124077.838149	170976.859445	-0.001270
1	Random Forest	552.294274	1472.020152	0.999926

The Random Forest model should substantially outperform the baseline because the housing market signal is strongly encoded in the lagged and market context variables.

A near perfect score should be interpreted carefully, since lagged target derived features and a random train test split can make prediction easier than a true future facing forecast task.

```
[41]: comparison_df = pd.DataFrame({
    "Actual_ZHVI": y_test.values[:10],
    "Baseline_Prediction": baseline_preds[:10],
    "RF_Prediction": rf_preds[:10]
})
```

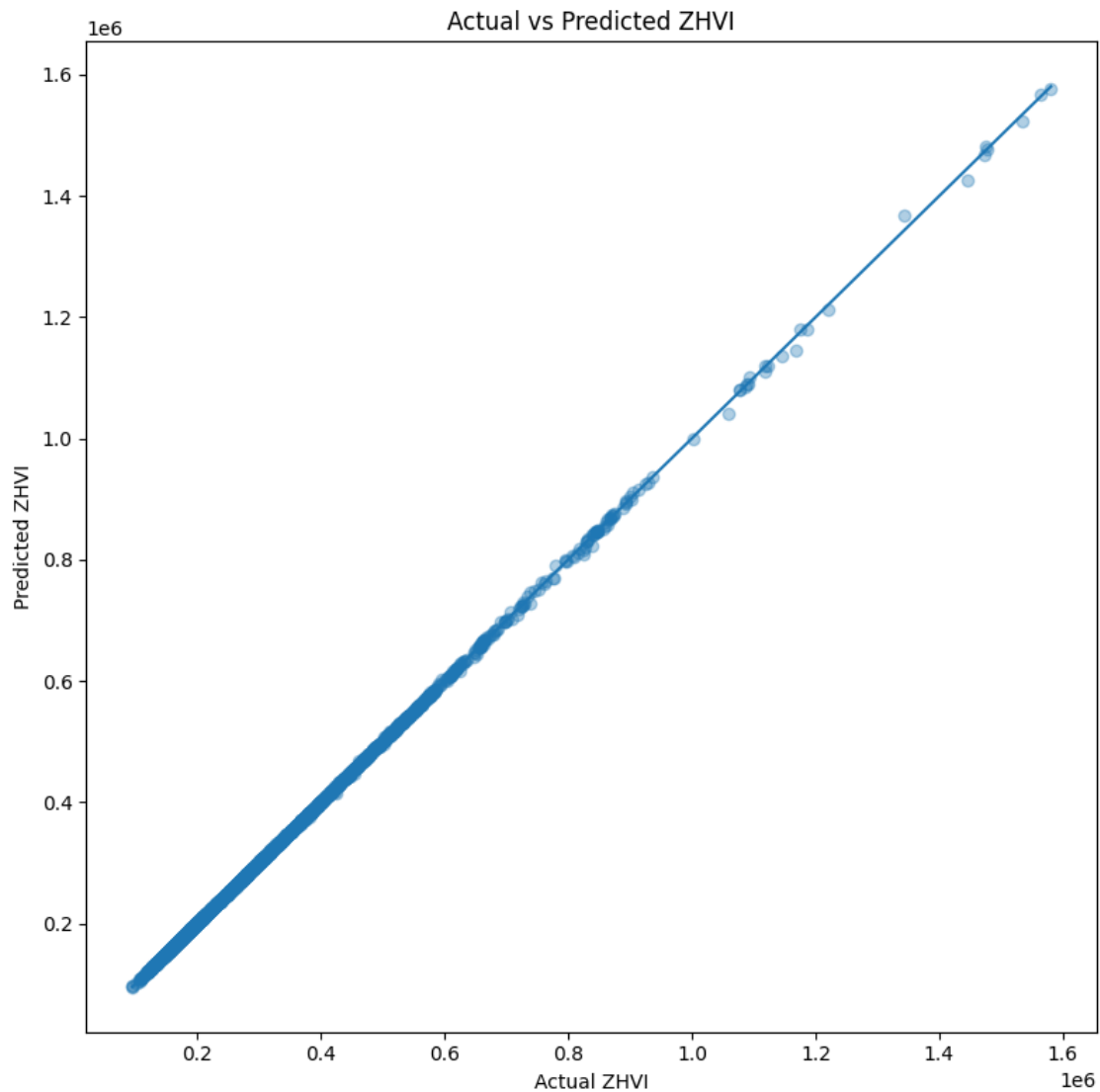
```
comparison_df
```

```
[41]:
```

	Actual_ZHVI	Baseline_Prediction	RF_Prediction
0	192320.039701	326929.338191	192085.661706
1	199973.096068	326929.338191	200244.094823
2	173588.027578	326929.338191	173551.486556
3	210487.171341	326929.338191	210626.358540
4	166221.923612	326929.338191	166174.019393
5	350575.186377	326929.338191	350426.329589
6	429152.900351	326929.338191	428984.785367
7	292276.567687	326929.338191	292066.393749
8	249028.051313	326929.338191	249553.045314
9	456323.944012	326929.338191	457202.242128

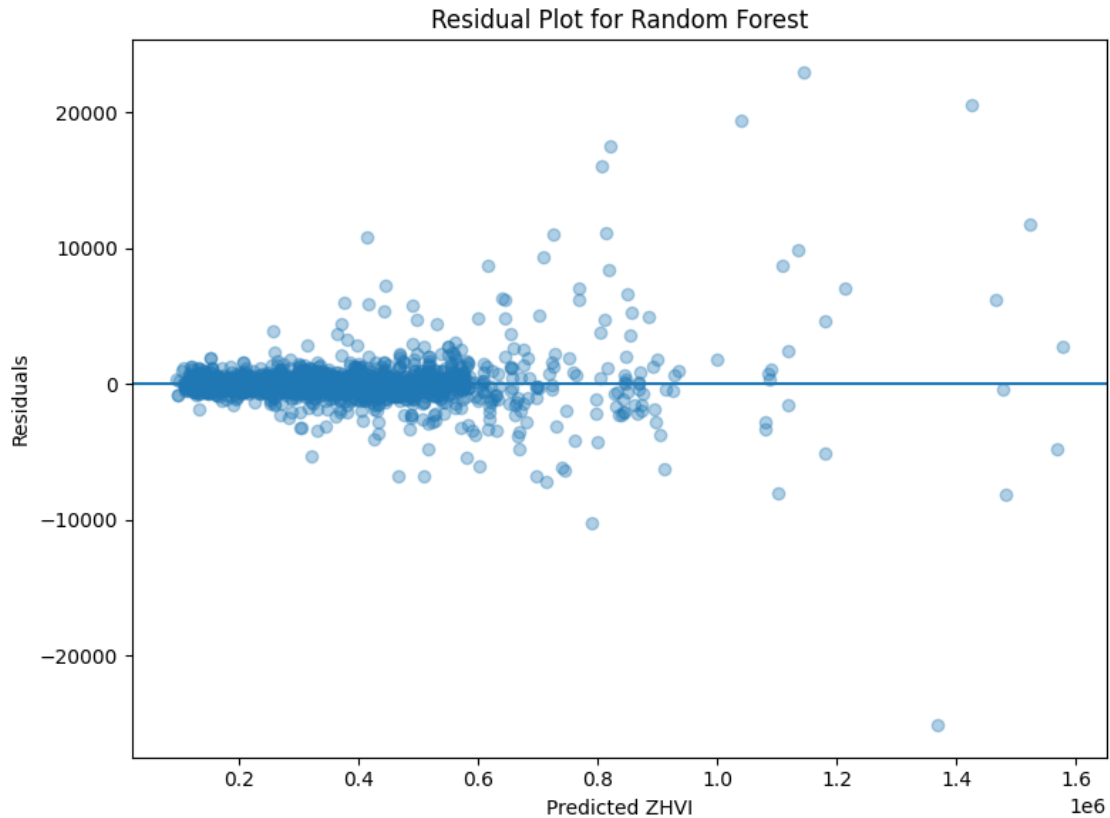
1.9 Visual Evaluation

```
[42]: plt.figure(figsize=(8, 8))
plt.scatter(y_test, rf_preds, alpha=0.35)
min_val = min(y_test.min(), rf_preds.min())
max_val = max(y_test.max(), rf_preds.max())
plt.plot([min_val, max_val], [min_val, max_val])
plt.xlabel("Actual ZHVI")
plt.ylabel("Predicted ZHVI")
plt.title("Actual vs Predicted ZHVI")
plt.tight_layout()
plt.show()
```



```
[43]: rf_residuals = y_test - rf_preds

plt.figure(figsize=(8, 6))
plt.scatter(rf_preds, rf_residuals, alpha=0.35)
plt.axhline(y=0)
plt.xlabel("Predicted ZHVI")
plt.ylabel("Residuals")
plt.title("Residual Plot for Random Forest")
plt.tight_layout()
plt.show()
```



1.10 Feature Importance

This section supports interpretability by showing which features contribute most strongly to the Random Forest model.

That is important from a business perspective because investors and decision makers need more than just a prediction. They need to understand the major drivers behind it.

```
[44]: rf_regressor = rf_model.named_steps["regressor"]
fitted_preprocessor = rf_model.named_steps["preprocessor"]

encoded_cat_names = (
    fitted_preprocessor
    .named_transformers_["cat"]
    .named_steps["onehot"]
    .get_feature_names_out(categorical_features)
)

all_feature_names = numeric_features + list(encoded_cat_names)

importance_df = (
    pd.DataFrame({
```



```

        "Feature": all_feature_names,
        "Importance": rf_regressor.feature_importances_
    })
    .sort_values("Importance", ascending=False)
)

importance_df.head(15)

```

```

[44]:

```

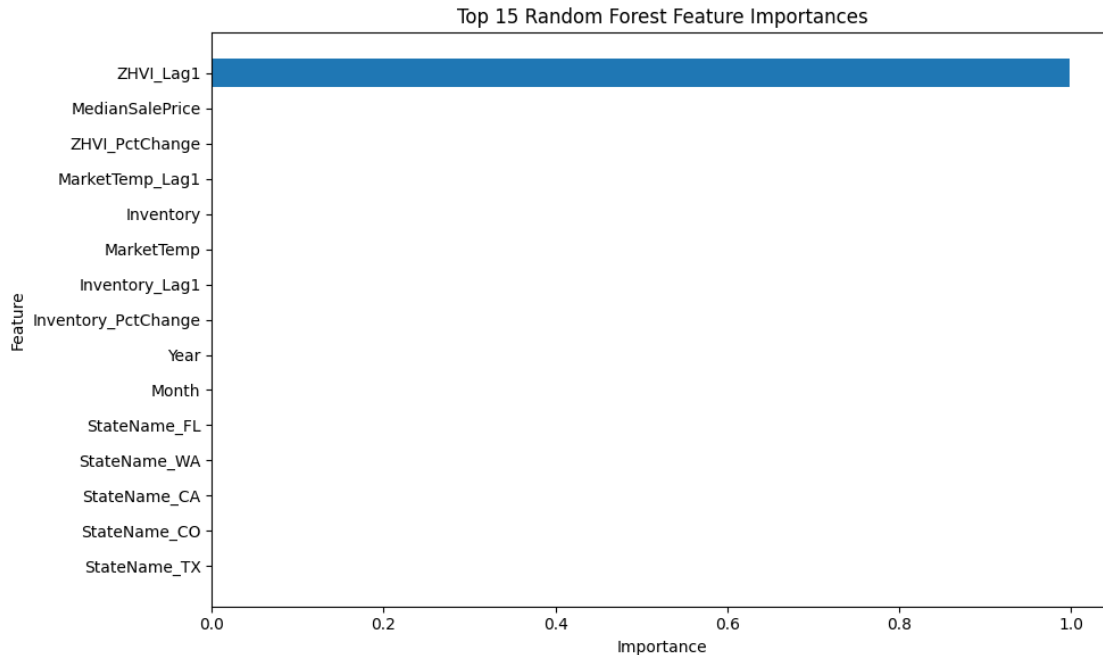
	Feature	Importance
5	ZHVI_Lag1	9.989051e-01
4	MedianSalePrice	8.165218e-04
8	ZHVI_PctChange	1.505483e-04
7	MarketTemp_Lag1	2.836113e-05
2	Inventory	2.806470e-05
3	MarketTemp	1.956305e-05
6	Inventory_Lag1	1.894771e-05
9	Inventory_PctChange	1.138155e-05
0	Year	1.029598e-05
1	Month	8.747637e-06
18	StateName_FL	4.945159e-07
50	StateName_WA	3.173276e-07
14	StateName_CA	3.161746e-07
15	StateName_CO	2.091137e-07
47	StateName_TX	1.946021e-07

```

[45]: top_features = importance_df.head(15).sort_values("Importance")

plt.figure(figsize=(10, 6))
plt.barh(top_features["Feature"], top_features["Importance"])
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.title("Top 15 Random Forest Feature Importances")
plt.tight_layout()
plt.show()

```



1.11 Key Findings

Based on the consolidated workflow, the project supports the following conclusions:

- Recent housing value history is one of the strongest predictors of current housing value.
- Inventory and market temperature provide useful complementary information about supply and demand conditions.
- The pipeline demonstrates that housing market indicators can be integrated into a scalable cloud based machine learning workflow for regional trend estimation.

1.12 Limitations

This final notebook is strong as a modeling demonstration, but several limitations should be acknowledged:

1. **Random train test split:** Because housing data is time ordered, a random split can introduce temporal leakage.
2. **Lagged target derived features:** Features such as prior ZHVI and ZHVI percentage change are highly predictive, but they can inflate performance if the evaluation strategy is not time aware.
3. **External variables not included:** Interest rates, unemployment, school quality, and other macroeconomic factors were not incorporated.

4. **Prepared dataset focus:** This notebook emphasizes the final integrated dataset rather than rebuilding every raw source transformation step from scratch.

1.13 Future Enhancements

The next improvements for this project are:

1. Replace the random split with a time series validation approach.
2. Add macroeconomic and demographic features to improve generalizability.
3. Automate the monthly ETL process using AWS Glue and scheduled SageMaker workflows.
4. Extend monitoring with model drift checks and retraining triggers.

1.14 Production Deployment Plan

A practical AWS deployment path for this project would be:

1. Store the prepared dataset and model artifacts in Amazon S3.
2. Package the final preprocessing plus Random Forest pipeline as a serialized model artifact.
3. Deploy the model through an Amazon SageMaker endpoint for on demand inference or use batch prediction for scheduled market reports.
4. Monitor data quality and model behavior over time.
5. Retrain the model periodically as new Zillow data becomes available.

```
[46]: # Optional: save the trained final model artifact locally
      joblib.dump(rf_model, "ADS508_GROUP6_final_rf_model.joblib")
      print("Saved model artifact as ADS508_GROUP6_final_rf_model.joblib")
```

Saved model artifact as ADS508_GROUP6_final_rf_model.joblib

1.15 Conclusion

This notebook consolidates the project into a single technical artifact suitable for the final repository.

It follows the overall structure used in the course examples while remaining aligned with the J&F Realty Insighters business problem, AWS workflow, and final design document.